QUANTUM SECURITY INTELLIGENCE

Security & Post-Quantum Readiness Audit Report

rust-secp256k1 @ fe8b3bec962f · rust-secp256k1 — public repository (passive review)

Organization	QSI
Report date	2026-10-28 19:28 UTC
Report version	1
Audit run	f030bcd6-5287-4f61-b078-ddc67276c23d · Document audit · 2026-10-28 19:28 UTC
Analyzer engine	qsi-engine 1.2.0
Risk profile	Crypto exchange, custodian or wallet
Content SHA-256	`2f9180a54b4c716991a4c8736399076b3da27e48b936ad8bd07be50fd10740f8`
Signature key (Ed25519, SHA-256 fingerprint; the signature is the separate file report.sig.json)	`5b3cf9c9bd0fa54447e2b5dfa7a6bab569bd0c666fa46949599e5c348d17083e`

Contents

1. Executive summary
2. Scope and boundaries
3. Methodology
4. Assets examined and not examined
5. Architecture overview
6. Overall security posture
7. Findings by severity
8. Detailed findings
9. Evidence
10. Post-quantum assessment
11. Limitations and assumptions
12. Remediation plan
13. Priorities for 30, 60 and 90 days
14. Verification after remediation
15. Tools used
16. Sources and normative documents
17. Appendices

This report presents automated and documented analysis results for the stated scope. It is not a legal opinion, a certification or an attestation of compliance with any standard.

Executive summary

4	0	0	4	0
Findings in total	Critical	High	Medium	Low

Classification	Count
Confirmed technical vulnerability	0
Known vulnerable component	0
Confirmed configuration error	0
Potential cryptographic risk	0
Long-term post-quantum risk	4
Improvement recommendation	0
Insufficient data	0

Top 5

ID	Severity	Detailed findings	Priority score
QSI-2026-06112	Medium	Quantum-vulnerable ECDSA: transaction and message signing (26 file(s)) preliminary	35.6
QSI-2026-06114	Medium	Quantum-vulnerable Schnorr (BIP-340): transaction and message signing (2 file(s)) preliminary	35.6
QSI-2026-06113	Medium	Quantum-vulnerable ECDH: role not determined (1 file(s)) preliminary	34.2
QSI-2026-06115	Medium	Quantum-vulnerable ECDSA: role not determined (2 file(s)) preliminary	34.2

Mosca's inequality (x + y > z): x = 10, y = 5, z = 8 → migration would finish too late under this scenario. z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.

Post-quantum maturity: Level 0 of 4 — no post-quantum cryptography. Next step: Inventory where classical public-key cryptography protects long-lived data and keys, and choose the first migration: usually a hybrid TLS key exchange and wrapped keys.

Scope and boundaries

Mode: Document audit

Repository: https://github.com/rust-bitcoin/rust-secp256k1 @ fe8b3bec962fc2c7052501de397fde58ef7142d1 — evidence links open the lines at this commit.

Supplied inputs

127 files, 4379620 bytes in total, 39 redaction(s); the per-file SHA-256 list is in appendix C.

Methodology

The audit was performed by the QSI engine 1.2.0. Each analyzer emits facts (rule, parameters, evidence); finding texts are generated from the rule catalog. Methodology sources are listed under "Sources and normative documents".

Prioritization uses an explainable weighted model (severity, exploitability, exposure, confidentiality/integrity/availability impact weighted by the "Crypto exchange, custodian or wallet" profile, asset criticality, urgency, ease of remediation) with a confidence multiplier. CVSS v3.1 is applied only to technical vulnerabilities with a defined vector. Long-term quantum risks use Mosca's inequality; their severity is capped at Medium and they are never presented as exploitable vulnerabilities.

Finding classes: confirmed technical vulnerability; known vulnerable component (a dependency version named in an advisory; whether the affected code is reachable is assessed separately, and a component confirmed by review becomes a vulnerability); confirmed configuration error; potential cryptographic risk; long-term post-quantum risk; improvement recommendation; insufficient data. Heuristic findings are marked preliminary; analyst decisions are shown next to the findings.

Coverage of checks

Area	Status	Count	Examples
dependencies	checked	2	Cargo.toml: 11 dependencies parsed (0 pinned); secp256k1-sys/Cargo.toml: 2 dependencies parsed (0 pinned)
source_code	checked	124	secp256k1-sys/build.rs: 68 lines scanned with 16 rule patterns and 122 registry patterns; secp256k1-sys/depend/check_uint128_t.c: 16 lines scanned with 16 rule patterns and 122 registry patterns; secp256k1-sys/depend/secp256k1/contrib/lax_der_parsing.c: 150 lines scanned with 16 rule patterns and 122 registry patterns
keys	error (not checked)	99	secp256k1-sys/depend/secp256k1/src/wycheproof/ecdsa_secp256k1_sha256_bitcoin_test.json line 134: public key could not be parsed (ValueError); secp256k1-sys/depend/secp256k1/src/wycheproof/ecdsa_secp256k1_sha256_bitcoin_test.json line 3600: public key could not be parsed (ValueError); secp256k1-sys/depend/secp256k1/src/wycheproof/ecdsa_secp256k1_sha256_bitcoin_test.json line 3636: public key could not be parsed (ValueError)
certificates_and_keys	checked	1	secp256k1-sys/depend/secp256k1/src/wycheproof/ecdsa_secp256k1_sha256_bitcoin_test.json: certificates/keys parsed
secrets	checked	7	src/ellswift.rs: 2 redacted value(s) not reported (comments, test modules, fixture declarations, placeholders, values the source annotates as published, or hex values not bound to a secret name); src/key/mod.rs: value(s) at line(s) 2162 (test_keypair_deserialize_serde) are declared inside a fixture and treated as test data, although the file itself is not test code; src/key/mod.rs: 3 redacted value(s) not reported (comments, test modules, fixture declarations, placeholders, values the source annotates as published, or hex values not bound to a secret name)
osv_advisories	not checked	1	no exactly pinned versions to query

Long lists are grouped here; the full per-file list is in the JSON report, signed with the same key.

Assets examined and not examined

Supplied inputs: 127 (input files: C)

Not checked

- keys: 99 (examples: secp256k1-sys/depend/secp256k1/src/wycheproof/ecdsa_secp256k1_sha256_bitcoin_test.json line 134: public key could not be parsed (ValueError); secp256k1-sys/depend/secp256k1/src/wycheproof/ecdsa_secp256k1_sha256_bitcoin_test.json line 3600: public key could not be parsed (ValueError); secp256k1-sys/depend/secp256k1/src/wycheproof/ecdsa_secp256k1_sha256_bitcoin_test.json line 3636: public key could not be parsed (ValueError))
- osv_advisories: 1 (examples: no exactly pinned versions to query)

Architecture overview

Static review of the source tree rust-secp256k1 at fe8b3bec962fc2c7052501de397fde58ef7142d1. 127 files analysed; of 216 files, 26 are test, fuzzing, benchmark or example code and were left out, 63 are not read by any analyzer, 0 exceed the per-file limit and 0 did not fit the total limit. No requests were sent to the owner's systems.

Overall security posture

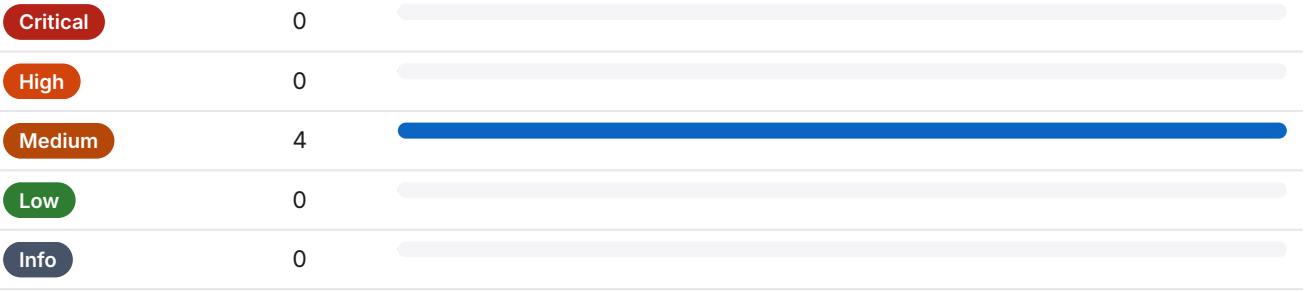
100
Indicative posture index (0–100)

100.0%
Quantum-vulnerable share of inventory

36
Cryptographic inventory

Derived from findings in scope (confirmed and potential issues weighted by severity). Indicative only; not comparable across differently scoped audits.

Findings by severity



ID	Detailed findings	Classification	Severity	Confidence
QSI-2026-06112	Quantum-vulnerable ECDSA: transaction and message signing (26 file(s))	Long-term post-quantum risk	Medium	Medium, preliminary
QSI-2026-06114	Quantum-vulnerable Schnorr (BIP-340): transaction and message signing (2 file(s))	Long-term post-quantum risk	Medium	Medium, preliminary
QSI-2026-06113	Quantum-vulnerable ECDH: role not determined (1 file(s))	Long-term post-quantum risk	Medium	Medium, preliminary
QSI-2026-06115	Quantum-vulnerable ECDSA: role not determined (2 file(s))	Long-term post-quantum risk	Medium	Medium, preliminary

Detailed findings

QSI-2026-06112 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable ECDSA: transaction and message signing (26 file(s))

Long-term post-quantum risk | Confidence: Medium | preliminary — requires further verification

56 reference(s) to ECDSA across 26 file(s), e.g. secp256k1-sys/build.rs:20. Role, judged from paths, function names and nearby code: transaction and message signing. A CRQC derives the private key from a public key that is already known (on chain after the first spend, in reused addresses): funds and approvals controlled by these keys need a migration path.

Affected asset	source-code:ECDSA:transaction_signing
Data source	secp256k1-sys/build.rs
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.2.0 · 2026-10-28 19:28 UTC
Priority score	35.6 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open secp256k1-sys/build.rs:20 and inspect the quoted lines; the evidence lists further files.
```

EVIDENCE

file_excerpt · secp256k1-sys/build.rs · line 20 · SHA-256 a5f2d41826e68b84… · File f28218d715a210cc…

```
   19 |     base_config
   20 |         .include("depend/secp256k1/")
   21 |         .include("depend/secp256k1/include")
```

file_excerpt · secp256k1-sys/build.rs · line 21 · SHA-256 4199c21bcb627930… · File f28218d715a210cc…

```
   20 |         .include("depend/secp256k1/")
   21 |         .include("depend/secp256k1/include")
   22 |         .include("depend/secp256k1/src")
```

file_excerpt · secp256k1-sys/build.rs · line 22 · SHA-256 92465db9e22b0cd0… · File f28218d715a210cc…

```
   21 |         .include("depend/secp256k1/include")
   22 |         .include("depend/secp256k1/src")
   23 |         .flag_if_supported("-Wno-unused-function") // some ecmult stuff is defined but not
used upstream
```

file_excerpt · secp256k1-sys/depend/secp256k1/contrib/lax_der_parsing.h · line 57 · SHA-256 25af7e242d92af0d… · File 9e3f5c6282938e3e…

```
56 |    files (such as tests.c) without the need to set an explicit -I flag,
57 |    which would be necessary to locate secp256k1.h. */
58 | #ifndef SECP256K1_H
```

file_excerpt · secp256k1-sys/depend/secp256k1/contrib/lax_der_parsing.h · line 59 · SHA-256 4c35a81e99a7eaf5… · File 9e3f5c6282938e3e…

```
58 | #ifndef SECP256K1_H
59 | #include <secp256k1.h>
60 | #endif
```

file_excerpt · secp256k1-sys/depend/secp256k1/contrib/lax_der_privatekey_parsing.h · line 34 · SHA-256 163cac24f6b60e79… · File 270578950fc61f08…

```
33 |    files (such as tests.c) without the need to set an explicit -I flag,
34 |    which would be necessary to locate secp256k1.h. */
35 | #ifndef SECP256K1_H
```

file_excerpt · secp256k1-sys/depend/secp256k1/contrib/lax_der_privatekey_parsing.h · line 36 · SHA-256 2741d1869779ddac… · File 270578950fc61f08…

```
35 | #ifndef SECP256K1_H
36 | #include <secp256k1.h>
37 | #endif
```

file_excerpt · secp256k1-sys/depend/secp256k1/include/secp256k1_ecdh.h · line 4 · SHA-256 1b629d577e7d70d5… · File 24a629250a48ff97…

```
3 |
4 | #include "secp256k1.h"
5 |
```

file_excerpt · secp256k1-sys/depend/secp256k1/include/secp256k1_ellswift.h · line 4 · SHA-256 1b629d577e7d70d5… · File 05e4a55ae2d4610f…

```
3 |
4 | #include "secp256k1.h"
5 |
```

file_excerpt · secp256k1-sys/depend/secp256k1/include/secp256k1_extrakeys.h · line 4 · SHA-256 1b629d577e7d70d5… · File 6d168046d730b7a6…

```
3 |
4 | #include "secp256k1.h"
5 |
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum signature schemes as chains adopt them.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-06114 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable Schnorr (BIP-340): transaction and message signing (2 file(s))

Long-term post-quantum risk | Confidence: Medium | preliminary — requires further verification

9 reference(s) to Schnorr (BIP-340) across 2 file(s), e.g. secp256k1-sys/src/lib.rs:1073. Role, judged from paths, function names and nearby code: transaction and message signing. A CRQC derives the private key from a public key that is already known (on chain after the first spend, in reused addresses): funds and approvals controlled by these keys need a migration path.

Affected asset	source-code:Schnorr (BIP-340):transaction_signing
Data source	secp256k1-sys/src/lib.rs
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.2.0 · 2026-10-28 19:28 UTC
Priority score	35.6 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open secp256k1-sys/src/lib.rs:1073 and inspect the quoted lines; the evidence lists further files.
```

EVIDENCE

file_excerpt · secp256k1-sys/src/lib.rs · line 1073 · SHA-256 efacee2282622bd9… · File 415aae64b2c5f2f5…

```
1072 |     )]
1073 |     pub fn secp256k1_schnorrsig_sign(
1074 |         cx: *const Context,
```

file_excerpt · secp256k1-sys/src/lib.rs · line 1086 · SHA-256 7155b9ee61a6b3b7… · File 415aae64b2c5f2f5…

```
1085 |     )]
1086 |     pub fn secp256k1_schnorrsig_sign_custom(
1087 |         cx: *const Context,
```

file_excerpt · secp256k1-sys/src/lib.rs · line 1099 · SHA-256 c0c60e7da096f64b… · File 415aae64b2c5f2f5…

```
1098 |     )]
1099 |     pub fn secp256k1_schnorrsig_verify(
1100 |         cx: *const Context,
```

file_excerpt · src/schnorr.rs · line 119 · SHA-256 b0ac4aa5cec5e41d… · File 259030c32d226d4f…

```
118 |             |secp: &Secp256k1<crate::AllPreallocated>| {
119 |                 ffi::secp256k1_schnorrsig_sign_custom(
120 |                     secp.ctx.as_ptr(),
```

file_excerpt · src/schnorr.rs · line 167 · SHA-256 3c8d77857e39e275… · File 259030c32d226d4f…

```
166 |             |secp: &Secp256k1<crate::AllPreallocated>| {
167 |                 ffi::secp256k1_schnorrsig_verify(
168 |                     secp.ctx.as_ptr(),
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum signature schemes as chains adopt them.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-06113 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable ECDH: role not determined (1 file(s))

Long-term post-quantum risk | Confidence: Medium | preliminary — requires further verification

4 reference(s) to ECDH across 1 file(s), e.g. secp256k1-sys/depend/secp256k1/src/bench.c:24. Role, judged from paths, function names and nearby code: role not determined. The code does not show what the keys protect; review decides the priority.

Affected asset	source-code:ECDH:general
Data source	secp256k1-sys/depend/secp256k1/src/bench.c
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.2.0 · 2026-10-28 19:28 UTC
Priority score	34.2 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open secp256k1-sys/depend/secp256k1/src/bench.c:24 and inspect the quoted lines; the evidence lists further files.
```

EVIDENCE

file_excerpt · secp256k1-sys/depend/secp256k1/src/bench.c · line 24 · SHA-256 6ec91ca104f465f4… · File 7032161ad890bf97…

```
23 | #ifdef ENABLE_MODULE_ECDH
24 |     printf("    - ECDH key exchange (optional module)\n");
25 | #endif
```

file_excerpt · secp256k1-sys/depend/secp256k1/src/bench.c · line 54 · SHA-256 b21ecd8d53157d4f… · File 7032161ad890bf97…

```
53 | #ifdef ENABLE_MODULE_ECDH
54 |     printf("    ecdh              : ECDH key exchange algorithm\n");
55 | #endif
```

file_excerpt · secp256k1-sys/depend/secp256k1/src/bench.c · line 68 · SHA-256 ae7392892e7a0840… · File 7032161ad890bf97…

```
67 |     printf("    ellswift_keygen   : ElligatorSwift key generation\n");
68 |     printf("    ellswift_ecdh     : ECDH on ElligatorSwift keys\n");
69 | #endif
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Classify each usage, record data lifetimes, and plan migration to Hybrid key establishment with ML-KEM (FIPS 203), e.g. X25519MLKEM768 in TLS 1.3 or mlkem768x25519-sha256 in SSH; introduce an abstraction layer for crypto-agility.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-06115 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable ECDSA: role not determined (2 file(s))

Long-term post-quantum risk | Confidence: Medium | preliminary — requires further verification

3 reference(s) to ECDSA across 2 file(s), e.g. src/lib.rs:362. Role, judged from paths, function names and nearby code: role not determined. The code does not show what the keys protect; review decides the priority.

Affected asset	source-code:ECDSA:general
Data source	src/lib.rs
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.

Analyzer	source_code 1.2.0 · 2026-10-28 19:28 UTC
Priority score	34.2 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open src/lib.rs:362 and inspect the quoted lines; the evidence lists further files.
```

EVIDENCE

file_excerpt · src/lib.rs · line 362 · SHA-256 ffad46a775c24c96… · File 41069109679af98f…

```
361 |     fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
362 |         write!(f, "<secp256k1 context {:?}, {}>", self.ctx, C::DESCRIPTION)
363 |     }
```

file_excerpt · src/lib.rs · line 416 · SHA-256 f7ed24bd3aef9099… · File 41069109679af98f…

```
415 |     #[cfg(feature = "rand")]
416 |     #[deprecated(since = "0.32.0", note = "use secp256k1::generate_keypair instead")]
417 |     pub fn generate_keypair<R: rand::Rng + ?Sized>(
```

file_excerpt · src/scalar.rs · line 151 · SHA-256 d1c2f06aed006e6e… · File 59603163ac5b2947…

```
150 |     fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
151 |         fmt::Display::fmt("the value is not a member of secp256k1 field", f)
152 |     }
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Classify each usage, record data lifetimes, and plan migration to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it; introduce an abstraction layer for crypto-agility.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

Evidence

ID	Type	Data source	SHA-256
QSI-2026-06112	file_excerpt	secp256k1-sys/build.rs (line 20)	a5f2d41826e68b84e6cc0e9f6c66bb2d8df138b21870fab304ea5baa3
QSI-2026-06112	file_excerpt	secp256k1-sys/build.rs (line 21)	4199c21bcb627930afb8759ea12971929ddaeea49d540dec5f3120dd9
QSI-2026-06112	file_excerpt	secp256k1-sys/build.rs (line 22)	92465db9e22b0cd0df7d8585a8505b1318dcb874a3c6015e59d2af2c1
QSI-2026-06112	file_excerpt	secp256k1-sys/depend/secp256k1/contrib/lax_der_parsing.h (line 57)	25af7e242d92af0d3c46ae8d8a48030e247c4bfdeba65068030e64eb4
QSI-2026-06112	file_excerpt	secp256k1-sys/depend/secp256k1/contrib/lax_der_parsing.h (line 59)	4c35a81e99a7eaf5ea7315346f3b8cacb0832f7c4a3b1a3d003ff0bd3
QSI-2026-06112	file_excerpt	secp256k1-sys/depend/secp256k1/contrib/lax_der_privatekey_parsing.h (line 34)	163cac24f6b60e798317e2383fb603da718e728035262dc9759d7cf3f
QSI-2026-06112	file_excerpt	secp256k1-sys/depend/secp256k1/contrib/lax_der_privatekey_parsing.h (line 36)	2741d1869779ddac4f151ac1740c338ba299b2bed7b98a38e6042acb0
QSI-2026-06112	file_excerpt	secp256k1-sys/depend/secp256k1/include/secp256k1_ecdh.h (line 4)	1b629d577e7d70d59383b1579315be3bf12eeca54bf16ca67903eeced
QSI-2026-06112	file_excerpt	secp256k1-sys/depend/secp256k1/include/secp256k1_ellswift.h (line 4)	1b629d577e7d70d59383b1579315be3bf12eeca54bf16ca67903eeced
QSI-2026-06112	file_excerpt	secp256k1-sys/depend/secp256k1/include/secp256k1_extrakeys.h (line 4)	1b629d577e7d70d59383b1579315be3bf12eeca54bf16ca67903eeced
QSI-2026-06114	file_excerpt	secp256k1-sys/src/lib.rs (line 1073)	efacee2282622bd9afdbd4d21fba13b4ae52c3b7950fd46c6eaad7f28
QSI-2026-06114	file_excerpt	secp256k1-sys/src/lib.rs (line 1086)	7155b9ee61a6b3b7a19c9ac3b631ad3edddee83c1b261e3bb09f225c0
QSI-2026-06114	file_excerpt	secp256k1-sys/src/lib.rs (line 1099)	c0c60e7da096f64bf928a080f61201ddc6e99b91eb09cbad8df405a0f
QSI-2026-06114	file_excerpt	src/schnorr.rs (line 119)	b0ac4aa5cec5e41d8282163f25ae8852222c94224505fe06a05e79b8e
QSI-2026-06114	file_excerpt	src/schnorr.rs (line 167)	3c8d77857e39e275e2dbfcd82048329024ab19596a4d5f0aeac8ee8c5

QSI-2026-06113	file_excerpt	secp256k1-sys/depend/secp256k1/src/bench.c (line 24)	6ec91ca104f465f4b861193bee0ed1f6d91c4074fb5085511ca06c7d
QSI-2026-06113	file_excerpt	secp256k1-sys/depend/secp256k1/src/bench.c (line 54)	b21ecd8d53157d4f920477f57713fa34b7fe6379df61daa64f27e4a7
QSI-2026-06113	file_excerpt	secp256k1-sys/depend/secp256k1/src/bench.c (line 68)	ae7392892e7a0840c3596f3442ca87927d89cc694ec91267d2a73193
QSI-2026-06115	file_excerpt	src/lib.rs (line 362)	ffad46a775c24c96bcfcbea7698aa07d993b31975376d7e818d1f3c45
QSI-2026-06115	file_excerpt	src/lib.rs (line 416)	f7ed24bd3aef909915629692e58bc72335aab84d369d315f1446ff543
QSI-2026-06115	file_excerpt	src/scalar.rs (line 151)	d1c2f06aed006e6ebf628ab453e342b3fd91780c66badcc3759826914

Post-quantum assessment

	Key establishment (x = how long data must stay confidential; harvest now, decrypt later)	Signatures (x = how long signatures/ keys must stay trusted)
Data/key lifetime x	10	10
Migration time y	5	5
Assumed years to CRQC z	8 (early)	8
x + y − z	7 → migration would finish too late under this scenario	7 → migration would finish too late under this scenario

Planning assumptions: x for key establishment, x for signatures, y are the defaults of the risk profile “Crypto exchange, custodian or wallet”, not the client's figures; with the client's actual lifetimes the verdict can change.

Post-quantum maturity: Level 0 of 4 — no post-quantum cryptography

Next step: Inventory where classical public-key cryptography protects long-lived data and keys, and choose the first migration: usually a hybrid TLS key exchange and wrapped keys.

Levels: 0 none; 1 present, not wired; 2 in use outside the critical paths; 3 hybrid in a critical path; 4 post-quantum by default in the critical paths (root of trust, key wrapping, channel key exchange). Judged from code and manifests.

What the quantum-vulnerable algorithms do

Role	Algorithms (files)	ID	First step
transaction and message signing	ECDSA (26), Schnorr (BIP-340) (2)	QSI-2026-06112, QSI-2026-06114	Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum signature schemes as chains adopt them
role not determined	ECDSA (2), ECDH (1)	QSI-2026-06113, QSI-2026-06115	Classify each usage, record data lifetimes, and plan migration to Hybrid key establishment with ML-KEM (FIPS 203), e.g. X25519MLKEM768 in TLS 1.3 or mlkem768x25519-sha256 in SSH; introduce an abstraction layer for crypto-agility

Roles are judged from file paths, function names and the code around each match; review confirms them. Root-of-trust keys and wrapped keys come first: they are hard to replace or can be recorded today.

z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.

Quantum status	Count
vulnerable (Shor)	36

Detailed findings: QSI-2026-06112, QSI-2026-06114, QSI-2026-06113, QSI-2026-06115

PQC readiness questionnaire

No readiness questionnaire was completed for this project.

Quantum algorithms relevant to this inventory

Quantum algorithm	Effect	Inventoried algorithms (rows)	What it means for this audit
Shor's algorithm — elliptic-curve discrete logarithms Q1: quantum computation offline, classical access to the victim	breaks	ECDH (3), ECDSA (30), Schnorr (BIP-340) (3)	Elliptic-curve keys are expected to fall before RSA keys of comparable classical strength. On blockchains, coins whose public key is already on chain (reused addresses, pay-to-public-key and Taproot outputs) are exposed first; plan the move to post-quantum signatures and avoid key reuse in the meantime.

Effects: breaks — replace the algorithm; weakens — larger keys or outputs restore the margin; included in standard parameters — no action at standard parameter sets; only with superposition queries — not applicable to ordinary deployments.

Limitations and assumptions

- This report presents automated and documented analysis results for the stated scope. It is not a legal opinion, a certification or an attestation of compliance with any standard.
- z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.
- keys — error (not checked): 99 (examples: secp256k1-sys/depend/secp256k1/src/wycheproof/ecdsa_secp256k1_sha256_bitcoin_test.json line 134: public key could not be parsed (ValueError); secp256k1-sys/depend/secp256k1/src/wycheproof/ecdsa_secp256k1_sha256_bitcoin_test.json line 3600: public key could not be parsed (ValueError); secp256k1-sys/depend/secp256k1/src/wycheproof/ecdsa_secp256k1_sha256_bitcoin_test.json line 3636: public key could not be parsed (ValueError))
- osv_advisories — not checked: 1 (examples: no exactly pinned versions to query)

Remediation plan

QSI-2026-06112 · P3 · 90 days — short-term risk reduction

Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-06114 · P3 · 90 days — short-term risk reduction

Add Schnorr (BIP-340) usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-06113 · P3 · 90 days — short-term risk reduction

Add ECDH usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.

Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-06115 · P3 · 90 days

short-term risk reduction

Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-06112 · P4 · 90 days

root-cause fix

Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it or a hybrid scheme in use.

QSI-2026-06114 · P4 · 90 days

root-cause fix

Plan migration of Schnorr (BIP-340) to (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.

Closure criteria	Re-audit shows or a hybrid scheme in use.

QSI-2026-06113 · P4 · 90 days

root-cause fix

Plan migration of ECDH to Hybrid key establishment with ML-KEM (FIPS 203), e.g. X25519MLKEM768 in TLS 1.3 or mlkem768x25519-sha256 in SSH (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows Hybrid key establishment with ML-KEM (FIPS 203), e.g. X25519MLKEM768 in TLS 1.3 or mlkem768x25519-sha256 in SSH or a hybrid scheme in use.

QSI-2026-06115 · P4 · 90 days

root-cause fix

Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it or a hybrid scheme in use.

Priorities for 30, 60 and 90 days

30 days

Priority	ID	Action	Type
None			

60 days

Priority	ID	Action	Type
None			

90 days

Priority	ID	Action	Type
P3	QSI-2026-06112	Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-06114	Add Schnorr (BIP-340) usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-06113	Add ECDH usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-06115	Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P4	QSI-2026-06112	Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-06114	Plan migration of Schnorr (BIP-340) to (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-06113	Plan migration of ECDH to Hybrid key establishment with ML-KEM (FIPS 203), e.g. X25519MLKEM768 in TLS 1.3 or mlkem768x25519-sha256 in SSH (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-06115	Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix

Verification after remediation

ID	Retest procedure
QSI-2026-06112	Re-run the inventory after refactoring.
QSI-2026-06114	Re-run the inventory after refactoring.
QSI-2026-06113	Re-run the inventory after refactoring.
QSI-2026-06115	Re-run the inventory after refactoring.

Tools used

Tool	Version
blockchain	1.0.0
certificates	1.1.0
chain_map	1.0.0
configs	1.1.0
dependencies	1.2.0
inventory_document	1.1.0
solidity	1.2.0
source_code	1.3.0
tls_code	1.1.0
tls_live	1.0.0
qsi-engine	1.2.0
OSV.dev API	v1

Sources and normative documents

Reviewed: 2026-10-13

1. C. Chevignard, P.-A. Fouque, A. Schrottenloher — Reducing the Number of Qubits in Quantum Discrete Logarithms on Elliptic Curves (IACR ePrint 2026/280) — IACR ePrint. https://eprint.iacr.org/2026/280 (revision of a EUROCRYPT 2026 paper: 3.12n + o(n) qubits; for n = 256, 1193 qubits and 22 runs of 2^38.98 Toffoli gates)
2. Common Vulnerability Scoring System v3.1: Specification Document — FIRST. https://www.first.org/cvss/v3.1/specification-document (final)
3. D. Litinski — How to compute a 256-bit elliptic curve private key with only 50 million Toffoli gates (arXiv:2306.08585) — arXiv. https://arxiv.org/abs/2306.08585 (preprint (2023-06-14): 6000 modules × 1152 physical qubits, one key per 10 minutes under its architecture assumptions)
4. FIPS 203 — Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM) — NIST. https://csrc.nist.gov/pubs/fips/203/final (final (August 2024))
5. FIPS 204 — Module-Lattice-Based Digital Signature Standard (ML-DSA) — NIST. https://csrc.nist.gov/pubs/fips/204/final (final (August 2024))
6. FIPS 205 — Stateless Hash-Based Digital Signature Standard (SLH-DSA) — NIST. https://csrc.nist.gov/pubs/fips/205/final (final (August 2024))
7. J. Proos, C. Zalka — Shor's discrete logarithm quantum algorithm for elliptic curves (Quantum Inf. Comput. 3(4), 2003; arXiv:quant-ph/0301141) — Quantum Information & Computation. https://doi.org/10.26421/QIC3.4-3 (peer-reviewed article)
8. M. Cain et al. — Shor's algorithm is possible with as few as 10,000 reconfigurable atomic qubits (arXiv:2603.28627) — arXiv. https://arxiv.org/abs/2603.28627 (preprint (2026-03-30): P-256 in a few days with 26,000 physical neutral-atom qubits; RSA-2048 one to two orders of magnitude longer)
9. M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018) — IEEE. https://doi.org/10.1109/MSP.2018.3761723 (peer-reviewed article; source of the x + y > z migration inequality)
10. M. Roetteler, M. Naehrig, K. Svore, K. Lauter — Quantum resource estimates for computing elliptic curve discrete logarithms (ASIACRYPT 2017) — IACR / arXiv. https://arxiv.org/abs/1706.06752 (peer-reviewed article: at most 9n + 2⌈log2 n⌉ + 10 qubits and 448 n^3 log2 n + 4090 n^3 Toffoli gates)
11. NIST IR 8547 — Transition to Post-Quantum Cryptography Standards — NIST. https://csrc.nist.gov/pubs/ir/8547/ipd (initial public draft (November 2024); proposes deprecating quantum-vulnerable public-key algorithms after 2030 and disallowing them after 2035 — dates may change in the final version)
12. P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997) — SIAM. https://doi.org/10.1137/S0097539795293172 (peer-reviewed article)
13. R. Babbush et al. — Securing Elliptic Curve Cryptocurrencies against Quantum Vulnerabilities: Resource Estimates and Mitigations (arXiv:2603.28846) — arXiv. https://arxiv.org/abs/2603.28846 (preprint (2026-03-30, v2 2026-04-15): 256-bit ECDLP with < 1200 logical qubits and < 90 M Toffoli gates (or < 1450 and < 70 M); < 500,000 physical qubits at 10^-3 error, minutes)

Appendices

A. Cryptographic inventory

Algorithm	Parameter	Quantum status	Classical status	Count	Locations	Examples
ECDH		vulnerable (Shor)	acceptable	3	3	secp256k1-sys/depend/secp256k1/include/secp256k1_ecdh.h; secp256k1-sys/depend/secp256k1/src/bench.c:24; src/ecdh.rs
ECDSA	secp256k1	vulnerable (Shor)	acceptable	24	24	secp256k1-sys/build.rs:20; secp256k1-sys/depend/secp256k1/contrib/lax_der_parsing.h:57; secp256k1-sys/depend/secp256k1/contrib/lax_der_privatekey_parsing.h:34
ECDSA		vulnerable (Shor)	acceptable	6	6	secp256k1-sys/depend/secp256k1/src/ecdsa.h; secp256k1-sys/depend/secp256k1/src/ecdsa_impl.h; secp256k1-sys/src/lib.rs:271
Schnorr (BIP-340)		vulnerable (Shor)	acceptable	3	3	secp256k1-sys/depend/secp256k1/include/secp256k1_schnorrsig.h; secp256k1-sys/src/lib.rs:1073; src/schnorr.rs:119

Long lists are grouped here; the full per-file list is in the JSON report, signed with the same key.

B. JSON

qsi.report/1 · SHA-256 2f9180a54b4c716991a4c8736399076b3da27e48b936ad8bd07be50fd10740f8

C. Input files

File	Size	SHA-256
Cargo.toml	3184	`f1840f6dfb58f6ca1bc442f730c97893c96648fbc84ca860cf4d31fb0c8e386`
secp256k1-sys/Cargo.toml	1619	`ff4c6efd6aeaf4f76348f7b2f3ad7477f62657998825eb07dc13745e6c4348e`
secp256k1-sys/build.rs	2705	`f28218d715a210ccf4a02a72a8b10ba1bc32e55aab62de87c620c071616e15c`
secp256k1-sys/depend/check_uint128_t.c	248	`c20d2a8c6f6991595840a78f0e72eb0b32cce41209e62d30599c238fd21d5f4`
secp256k1-sys/depend/secp256k1/contrib/lax_der_parsing.c	3855	`d59c736417fbed06df5db4f476caf5643b55ecc5c950596ee3b80462dfb5285`
secp256k1-sys/depend/secp256k1/contrib/lax_der_parsing.h	4290	`9e3f5c6282938e3e141e911381ec6aebb321ad5a6d03b1e9e7a921548b6b51c`
secp256k1-sys/depend/secp256k1/contrib/lax_der_privatekey_parsing.c	5182	`c729549dd004b2631c72ddd9083b370886de6bfde0f06fad01eddafdc8da27b`
secp256k1-sys/depend/secp256k1/contrib/lax_der_privatekey_parsing.h	4056	`270578950fc61f0875a252f809a718a9a414754e17fc070846d577113e11e3e`

secp256k1-sys/depend/secp256k1/include/secp256k1.h	43026	d39c553b7903dfaec2c0947e33ed33eafbdfa09b3ef4a59fd4eb32ac959f370
secp256k1-sys/depend/secp256k1/include/secp256k1_ecdh.h	2710	24a629250a48ff976ddff7c56f0b4e3ce47ea49b97105accc2e41a4b5eaa017
secp256k1-sys/depend/secp256k1/include/secp256k1_ellswift.h	9397	05e4a55ae2d4610f8f832472824bc7c6dd9a96e347e9f956a5320b39b6f7a24
secp256k1-sys/depend/secp256k1/include/secp256k1_extrakeys.h	11490	6d168046d730b7a69f50e7793bf4254f6cb56c22002a3249836b41c6ea907a3
secp256k1-sys/depend/secp256k1/include/secp256k1_musig.h	29265	4fc94ae766ba1edb13003431948183fece6328ba682a3bea9496998184b219c
secp256k1-sys/depend/secp256k1/include/secp256k1_preallocated.h	6306	6ca991814860e7448cc67d8501a39b9327fba65a5501bf79951d3ac861e79a4
secp256k1-sys/depend/secp256k1/include/secp256k1_recovery.h	4922	f87789710c856ee08d6f04fa0809343cbdb508a9045580cf91ffe4aaf717be8
secp256k1-sys/depend/secp256k1/include/secp256k1_schnorrsig.h	8487	d76fff4bac7c0214df645179031dfe75d56df83c17bcdf646d0aa189251f20d
secp256k1-sys/depend/secp256k1/src/assumptions.h	4677	cbad7817c315f29783e2fa21f6477888b20a262672f674bc94c5858d3309806
secp256k1-sys/depend/secp256k1/src/bench.c	10281	7032161ad890bf972879b77ccc970b0b07fb39d75c4043617c2b030ca410198
secp256k1-sys/depend/secp256k1/src/bench.h	5088	f1739e0f0a8eb444edb516cbf42faa4c503145b81eaa857c0bf2f35459eadf3
secp256k1-sys/depend/secp256k1/src/bench_ecmult.c	15847	e92a968a6637adc0ac4a4ef0e013723f103b52e2a52116c48e7962f4f7fd1de
secp256k1-sys/depend/secp256k1/src/bench_internal.c	18104	7de9942375b76d01842227e9adfebb5d0511acd60068548115e06a7e6d2514e
secp256k1-sys/depend/secp256k1/src/checkmem.h	4640	338d267031b23e83133614ed1ad23e68480ba9f837c497155c326d08322d3c1
secp256k1-sys/depend/secp256k1/src/ecdsa.h	1344	011f5744a12548a5c333e6ea3a3d0e43b7f90a698955f7452e948b552abb19c
secp256k1-sys/depend/secp256k1/src/ecdsa_impl.h	11302	c6f44808c93278a1bd42aea9c6cc66bafc2df90ad4b09bf25ace1efa0965b11
secp256k1-sys/depend/secp256k1/src/eckey.h	1323	b1f23329bb6abe30e52f335975ba7617564ad10dccd580a894ffd0bef965e46
secp256k1-sys/depend/secp256k1/src/eckey_impl.h	3843	5f73e180882c209f28352612dd996136abdb102e93d9e3f8e2478888db20e46
secp256k1-sys/depend/secp256k1/src/ecmult.h	2819	1aedbc9abb1bce054768b366f886c40e0ac0cf535b4deb0859cf858c692419b
secp256k1-sys/depend/secp256k1/src/ecmult_compute_table.h	1153	7f8b7cb544b64c46f2d9744c5fdecf5115a0ee898f3be86c1ab666567e6661a

secp256k1-sys/depend/secp256k1/src/ecmult_compute_table_impl.h	2143	a3427b408943cfc6c1a4edefb9e9ee565f8cffe3728fe9e2396e7ff4163b2ec
secp256k1-sys/depend/secp256k1/src/ecmult_const.h	1454	6b4c471e779bc5c6eeb6014250bfb582e4713437208c1d8d8024cef6dce148e
secp256k1-sys/depend/secp256k1/src/ecmult_const_impl.h	20379	8838775cdd25cf65e253dc226e9ef0573a8c8cf58efb0c1e8a8fda4fbb9f943
secp256k1-sys/depend/secp256k1/src/ecmult_gen.h	5528	02b5f57bbcd49da5dd3cb385e326ceef3eebdca190dfe6ec5d439417691501c
secp256k1-sys/depend/secp256k1/src/ecmult_gen_compute_table.h	702	7dbc14514dbc0228998a73078865ce5bb1913c56f9ae1ec28be25511d41f018
secp256k1-sys/depend/secp256k1/src/ecmult_gen_compute_table_impl.h	5015	08fe63e5adbb6e67a55fb66fc2d14554c8643db49c60e9d279eea57708551c9
secp256k1-sys/depend/secp256k1/src/ecmult_gen_impl.h	17227	77fc665701acc1aaae6b61a73c4a05431be6a673f93c31d77bd77e9c9bcb891
secp256k1-sys/depend/secp256k1/src/ecmult_impl.h	36899	aabc88f8acaaf07b5e931e58535a5e15af86f30cc128c18b6c49714f548010f
secp256k1-sys/depend/secp256k1/src/field.h	16635	1a4ad5e0a6047cea3d54d573093756e4484d24e5d62e70a6a4bf941c529125a
secp256k1-sys/depend/secp256k1/src/field_10x26.h	2358	47f9d068809137d2ffb9119bf1cfea3280e0aeb11978feea0ac7802be37a010
secp256k1-sys/depend/secp256k1/src/field_10x26_impl.h	53112	02550b9ca48ded198f3aa8db9d8c6eee0bb6be585d82caa93f1413e382582c2
secp256k1-sys/depend/secp256k1/src/field_5x52.h	2400	9b6ab1a6c79b776d4b082ee127ce471f355e255bf5108effb599a96b9029ff7
secp256k1-sys/depend/secp256k1/src/field_5x52_impl.h	20326	2e3d35fcffced09bb6ac898760af75fcf26c6cb4260e1a89c2340fc516788fc
secp256k1-sys/depend/secp256k1/src/field_5x52_int128_impl.h	12391	59ed9576d6deb63c100b26019a33d24c230a7d136b956f67133e6877208530e
secp256k1-sys/depend/secp256k1/src/field_impl.h	16223	0afd349a2cbe8f9846c4c9651f09c982fb16334eea6ff8cca1971a87cf810ae
secp256k1-sys/depend/secp256k1/src/group.h	13118	ba3b2da5ab27366dd92b636e7d8dca9249c2db923c4239d7189dd958c673476
secp256k1-sys/depend/secp256k1/src/group_impl.h	38781	8aefc6d6eeab3d16aa83ab3355596b2dcf04c921ede1f4e2b09f825dd2cb6d3
secp256k1-sys/depend/secp256k1/src/hash.h	2196	2a78cc73f4ecf25179f859c7778e0c9eb2d7a4ac03e2c13d8fd3480fa0bebd0
secp256k1-sys/depend/secp256k1/src/hash_impl.h	14582	d152eb1b1578edab2ecf3ada77aa185ea52020c1bdcbc9eb4c393fdd8295a77
secp256k1-sys/depend/secp256k1/src/hsort.h	1705	bd108b51741b926781cb0a3ff2a054b61db2296f09857dfb2150fd661ace6cf

secp256k1-sys/depend/secp256k1/src/hsort_impl.h	5038	6a794299c0fa53600b8bce5c4c463586cdbd204fe2eaad469e3eb9c12a9c4d6
secp256k1-sys/depend/secp256k1/src/int128.h	4012	8435767d08876d419b3b743dd5ae31b07e0b74218df1be45462ea487015d84c
secp256k1-sys/depend/secp256k1/src/int128_impl.h	376	5683c02dfc495639da259d80624711c1c449bbaee6186729e399e713a28b471
secp256k1-sys/depend/secp256k1/src/int128_native.h	601	c28a470a02e1fd799f2cf8228ffe1731b3e0d2be191978355697a235e26a680
secp256k1-sys/depend/secp256k1/src/int128_native_impl.h	3271	3c1cbd5272d9b62783acd262d5807aee79d6d386f4aba888eaf906512c82d4e
secp256k1-sys/depend/secp256k1/src/int128_struct.h	259	b8eed83cae0de920af460eefdb26d796862a8dd36759869db76a72db262c298
secp256k1-sys/depend/secp256k1/src/int128_struct_impl.h	8434	1c09fafe1861c7ce5e524a92e561afd6f1525dec86b154d7abb39d09d033431
secp256k1-sys/depend/secp256k1/src/modinv32.h	1995	cc7da45d20d976a9753a7926a04fb694a151b35329d8d1ebbd008febddf9ad8
secp256k1-sys/depend/secp256k1/src/modinv32_impl.h	34692	248a5c185fdd1b34c22e8d0f12d1c3d80211aa37bb83178fde22abef062c988
secp256k1-sys/depend/secp256k1/src/modinv64.h	2099	db15aae18793d2bd401f9e9de5d8b22c513787f85aec694e8b367b9e0cb69ef
secp256k1-sys/depend/secp256k1/src/modinv64_impl.h	40233	8cbe489cf14cbbd8fe916ad91d02ac99d37bb442f9f29b9efeff73782190970
secp256k1-sys/depend/secp256k1/src/modules/ecdh/bench_impl.h	1935	a57875b66001de606592b8b993e89c2a0d81334b1fb7c2a5e7c3c61aff3b420
secp256k1-sys/depend/secp256k1/src/modules/ecdh/main_impl.h	2822	53241d7caec889c4f41c4ee71cda197214ff31caf20459425a6af86d48c07b4
secp256k1-sys/depend/secp256k1/src/modules/ellswift/bench_impl.h	4529	30b383bce338073942ecd69ef8d3a21b302dc8e907f15bb70b8214b72d8a587
secp256k1-sys/depend/secp256k1/src/modules/ellswift/main_impl.h	28694	bed144aea3f7559f294e57d9e3062386d1c12de7ebf48a5d0ccebde9aece273
secp256k1-sys/depend/secp256k1/src/modules/extrakeys/main_impl.h	11241	27b18e35f1200c70329123c9dc6bc2ae54de2ea15f7f7671a80fb1cdb2a02b3
secp256k1-sys/depend/secp256k1/src/modules/musig/keyagg.h	1408	079c5b03c77422b29a33294aea99b9c26c22fb53e89a1d7152afe0d05a68eec
secp256k1-sys/depend/secp256k1/src/modules/musig/keyagg_impl.h	12327	05b8ee77127f75455ab1e169579a9e86f43fcaafc0cb7182b60b44449695be5
secp256k1-sys/depend/secp256k1/src/modules/musig/main_impl.h	426	9ec7e6082aa496353466769bb9f7705c32caf8d060b0c92815cb37f6c426565
secp256k1-sys/depend/secp256k1/src/modules/musig/session.h	941	c12f5ec3df77df5ad3f4fcdfe99c8545c574825680c333c12e086238696ce0e

secp256k1-sys/depend/secp256k1/src/modules/musig/session_impl.h	34842	168e7cc3ac954ef9927711879f1cc44c788521057402695c723c14509047bb4
secp256k1-sys/depend/secp256k1/src/modules/musig/vectors.h	37797	c7cd9dc3804d864d17e43c5b00335a405a3bc4a30861838ac4599a213231e13
secp256k1-sys/depend/secp256k1/src/modules/recovery/bench_impl.h	2400	b6a1f35f2fee1653a5e2c2b1f88a6ec3bb59aa421267ea8cb52d0e254990405
secp256k1-sys/depend/secp256k1/src/modules/recovery/main_impl.h	6985	197bdb189d471cfb9123cab1b1beb5118e89985e4688d00781fd554aa6fa3c0
secp256k1-sys/depend/secp256k1/src/modules/schnorrsig/bench_impl.h	4137	b344d0ba298d435a80868003e56bee1221736a787964a8b56e053683cc32f84
secp256k1-sys/depend/secp256k1/src/modules/schnorrsig/main_impl.h	11292	b55b1d634b9bcac79c10278b2d1407cc0bcad13c813bc65ff0deccd6f117adb
secp256k1-sys/depend/secp256k1/src/precompute_ecmult.c	4134	eab942df74b984eb0cb55b924344391a399bb07472886501b19776e9554789
secp256k1-sys/depend/secp256k1/src/precompute_ecmult_gen.c	3939	77e375055b057465e20bb0bc703f46203c16cb41b98ce76856b53b88a6116fc
secp256k1-sys/depend/secp256k1/src/precomputed_ecmult.c	2409228	44c8513c966cd6f49ecc57a5a623dbb2e9f17d70fa09039fce0738d2de515c7
secp256k1-sys/depend/secp256k1/src/precomputed_ecmult.h	1665	cee6422b15bd9fe22fd5306613cf2cabd690490bebe3afd794bc8961e92191e
secp256k1-sys/depend/secp256k1/src/precomputed_ecmult_gen.c	259735	feefab4d4c735d9fd3e76271eeeda476e9859022e5c845bd8a3f83947cf3838
secp256k1-sys/depend/secp256k1/src/precomputed_ecmult_gen.h	1013	c045bde13bd99de253b19351959d1c0411f0398a53c15e5c2e20a5a0f05f1e9
secp256k1-sys/depend/secp256k1/src/scalar.h	6075	d3561ecba79556e0d7d2a3f0cf93398bd1f2353441cb0abdd9f8fc49ad67c04
secp256k1-sys/depend/secp256k1/src/scalar_4x64.h	811	1f7e75c03494ed55e6466b84507e3f240ad5616bc85a8e1f9f46daaa3150fd3
secp256k1-sys/depend/secp256k1/src/scalar_4x64_impl.h	35169	183eaa00def71b4b240eda3354968de160b8908b4a891399d75622e5939699
secp256k1-sys/depend/secp256k1/src/scalar_8x32.h	735	9f77522d7fd95caf61693492fc2883800722f2978f64fcf04420a3daec0d13e
secp256k1-sys/depend/secp256k1/src/scalar_8x32_impl.h	30216	4ef4a71103799cf6192dbcd6e49da527ec022abb5380147cda44761e66d906f
secp256k1-sys/depend/secp256k1/src/scalar_impl.h	12744	c4c79e9d73303c958128906625a1c96fe35e06fcd7a2c0dea282c827088bc4c
secp256k1-sys/depend/secp256k1/src/scalar_low.h	1145	4138194717181b6227fc03529f1e5fe54789515f0d3ee83b5c258b28fdb2fd9
secp256k1-sys/depend/secp256k1/src/scalar_low_impl.h	6411	570e203ca151e15563312a753715780b01d23a5a1f171c5511f71af38b818ab

secp256k1-sys/depend/secp256k1/src/scratch.h	2182	aa9c7b6d2e1e68d17dd2e70d282bd0227302bae7aa7396062da205429009c96
secp256k1-sys/depend/secp256k1/src/scratch_impl.h	2867	02c34bdf19ebcaf23d24b0b6d4b9aae31809d3edec285386bddbef3cf7b2ea7
secp256k1-sys/depend/secp256k1/src/secp256k1.c	30671	39e452b52a252c9389ae99ecd26489f34122018ad053b566ebf213ad874af47
secp256k1-sys/depend/secp256k1/src/selftest.h	1351	cca836fd26cb12b60a66ad808c9e89bd42a45c3220626def81437f7fbae6f97
secp256k1-sys/depend/secp256k1/src/testrand.h	1886	b1708cdab67d44311b97af6f77c719f6dfc03d8c53e5e686bc1d9fd967a4968
secp256k1-sys/depend/secp256k1/src/testrand_impl.h	5805	5303c670a157d2857c1f5de4567f73d04b2f5a37434eae1fcaabe00597dab57
secp256k1-sys/depend/secp256k1/src/testutil.h	4603	a2f705686ab565024949f5a4018ccb978b4c9e0eb11fec58a771b1746700965
secp256k1-sys/depend/secp256k1/src/util.h	14727	224fda59fc8ed42d1b259dfef2ecdf130a3e4c126cced79d0d136252151815 3
secp256k1-sys/depend/secp256k1/src/util_local_visibility.h	402	d47d4a58f40157ab2a298151f366c5ebec8f7a1e7aaba88e36b530b1c01584c
secp256k1-sys/depend/secp256k1/src/wycheproof/ecdsa_secp256k1_sha256_bitcoin_test.json	299043	0a0b8066df1e1b6576f2e2da0b1241170e23a5beca7ed127647c4847523569 3
secp256k1-sys/depend/secp256k1/tools/symbol-check.py	2426	fc152ef1c87a7c0c1d6487fe3f451068f42efe3c1a7205571404226c2a364f2
secp256k1-sys/depend/secp256k1/tools/wycheproof_utils.py	427	9f2c2368953eeb3c9b7c3b31a807cde76b3106df93fbb975993638d5eb1a1e8
secp256k1-sys/src/lib.rs	70718	415aae64b2c5f2f51644db0273decde6dedf092e159d583810f1bd439f759d7
secp256k1-sys/src/macros.rs	3221	5cc2d18ba578ab1bd5f24cda2e5d04bec238e0ac5d615845b5f49f13aed8671
secp256k1-sys/src/recovery.rs	7321	0b3128bbdb9d7b6896d23fa70a495644627b22fbcbcea2013bfc12d5cab4988
secp256k1-sys/src/types.rs	3312	5d51de698119d62dae9f7402b415080a4bb3efcf3105cf1d11881fa48474e2c
secp256k1-sys/wasm/wasm-sysroot/stdio.h	0	e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b85
secp256k1-sys/wasm/wasm-sysroot/stdlib.h	0	e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b85
secp256k1-sys/wasm/wasm-sysroot/string.h	236	fbfdac6ad0b66450c634081c5270b724179fd2b5dd615275489b3c22d275268
secp256k1-sys/wasm/wasm.c	710	804c35ac2746282ad01c6bf1bf331ff4bb17843f14750308580575215caaeb6
src/constants.rs	2446	b731982fed3921dd7b391c5185cfb80a9bac3519147fe760fe9af10d012e3b5
src/context/internal_nostd.rs	6529	6a41454cde81eb9f37b40f7f35977575766c89d3a275ad616f689178424045c
src/context/internal_std.rs	3291	ace7fc791d4ecab0d3e171ae20f3e377f23b6eb0a5ea8cf700bd99382db15a5

src/context/mod.rs	16342	8c2ad10e9f9a1a23a9ea56ebabaa4e7c5a12c41bf739addff45a1b394ae3b1b
src/context/spinlock.rs	7872	0f63af558b09bcc4a563853efa343628c3de044e86a6aa1fe9f412904f1f0d3
src/ecdh.rs	8625	b7a382d0fbefc1944aecab86e5fa62e1fcb07d16ab97b89a1c5352357428009
src/ecdsa/mod.rs	19911	c2501fabd9e06440ba379326e1179f0a1501d86b6c8025278e081bfc7e70359
src/ecdsa/recovery.rs	16666	acec0d5ab6859c999ddcf8b441a6e5014022e9e91b310443f4be9d7f0654fe0
src/ecdsa/serialized_signature.rs	9043	f7083c232563eed68b246c14d7b4329152b41a1ab5cad21053cf7d80a19cfd5
src/ellswift.rs	46614	0f8263f7a08819e9cee525235766c818379d5005ff04a7e7b9be3e48b88b751
src/key/mod.rs	75790	811fd2982f101aa42c8157f37db01cb4b4e13850686cdb596319e20b761c8e1
src/key/secret.rs	11958	df49878912b209974289c9d479c12ab2b19c1a6d0638539ad5dd62d1401936e
src/lib.rs	42297	41069109679af98f2c060205d76170ecba86ad5f26ad30c4894d1b751b09d6a
src/macros.rs	4164	31ece8ad2fa96786df7cc5498e455f454dee353c6a0383c2a282f87a034d0b7
src/musig.rs	69169	a22fd3466a745b4fad5e685c04a0cda15648102fc580db8b29eb1be41414cf6
src/scalar.rs	5109	59603163ac5b2947abbbe913d83a8950713a87afaad29aa929c821e5fc0d21e
src/schnorr.rs	31606	259030c32d226d4f1ef6ccbe7426f0f86874ec494aed531855f743140af02a2
src/secret.rs	7120	b9f4f2ee7314d6ba80e8ecc1d50b3a9050671acc2e2649109e7831db5977f8a
src/serde_util.rs	3017	a3c07e7486b54c50e5fbd1c47391cd84f3debfbad051036ee5e42de920620d9